COMMUNITY PROJECT REPORT

SMARTGARBAGE CHINTALAVALASA

Community-Based Smart Waste Management
and Digital Governance System

Submitted by

MOPADA JAGANMOHAN (24331A4441)
LATCHUPATULA RESHMA (24331A4434)
PATI NARASIMHA MURTHY (24331A4446)
KADA AUGUSTTN PAUL KUMAR (24331A4426)

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING
(Data Science)

Under the esteemed Guidance of

Mrs. S. Nikhila
Assistant Professor

DEPARTMENT OF DATA ENGINEERING
MAHARAJ VIJAYARAM GAJAPATHI RAJ COLLEGE OF ENGINEERING (Autonomous)
(Approved by AICTE, New Delhi, and permanently affiliated to JNTUGV, Vizianagaram)
Vijayaram Nagar Campus, Chintalavalasa, Vizianagaram-535005, Andhra Pradesh

October, 2026

CERTIFICATE

This is to certify that the project entitled "SmartGarbage Chintalavalasa — Community-Based Smart Waste Management and Digital Governance System" is the bonafide work carried out by Mopada Jaganmohan (24331A4441), Latchupatula Reshma (24331A4434), Pati Narasimha Murthy (24331A4446), and Kada Augusttn Paul Kumar (24331A4426), of B.Tech V Sem CSE-DS, M.V.G.R. College of Engineering (Autonomous), Vizianagaram, during the year 2026-2027, in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology and that the project has not formed the basis for the award previously of any degree or any other similar title.

Signature of Project Guide
Mrs. S. Nikhila
Assistant Professor
Department: Data Engineering

Signature of Head of the Department
Dr. Jyothi
Head of the Department
Department: Data Engineering

DECLARATION

We hereby declare that the work done on the dissertation entitled "SmartGarbage Chintalavalasa — Community-Based Smart Waste Management and Digital Governance System" has been carried out by us and submitted in partial fulfilment for the award of credits in Bachelor of Technology in Computer Science and Engineering (Data Science) of M.V.G.R College of Engineering (Autonomous) and affiliated to JNTUGV, Vizianagaram. The various contents incorporated in the dissertation have not been submitted for the award of any degree of any other institution or university.

MOPADA JAGANMOHAN (24331A4441)

LATCHUPATULA RESHMA (24331A4434)

PATI NARASIMHA MURTHY (24331A4446)

KADA AUGUSTTN PAUL KUMAR (24331A4426)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide for their invaluable guidance and support as our mentor throughout the project. Their unwavering commitment to excellence and constructive feedback motivated us to achieve our project goals. We are greatly indebted to them for their exceptional guidance.

Additionally, we extend our thanks to Prof. P.S. Sitharama Raju (Director), Dr. Y.M.C. Shekar (Principal), and Dr. Jyothi (Head of the Department) for their unwavering support and assistance, which were instrumental in the successful completion of the project.

We also acknowledge the dedicated assistance provided by all the staff members in the Department of Data Engineering. Finally, we appreciate the contributions of all those who directly or indirectly contributed to the successful execution of this endeavor.

MOPADA JAGANMOHAN (24331A4441)
LATCHUPATULA RESHMA (24331A4434)
PATI NARASIMHA MURTHY (24331A4446)
KADA AUGUSTTN PAUL KUMAR (24331A4426)

ABSTRACT

Waste management in semi-urban Indian communities like Chintalavalasa, Andhra Pradesh, is plagued by informal collection schedules, paper-based complaint tracking, and overflowing bins with no accountability.

SmartGarbage is a community-based smart waste management system built for the Chintalavalasa Gram Panchayat. It lets citizens check collection schedules, report missed pickups with photos and GPS, and track complaint resolution in real time. Administrators monitor operations through a live dashboard, assign workers, and view ward-level analytics.

The system uses IoT-enabled smart bins to monitor fill levels and predict overflow risk, enabling proactive collection. A pay-as-you-throw billing mechanism charges residents for non-segregated waste, encouraging proper segregation. Green Points gamification rewards citizens for consistent segregation behaviour.

For low-connectivity areas, Progressive Web App capabilities allow offline complaint filing with automatic sync. Bilingual support in English and Telugu ensures wide accessibility.

The prototype is deployed and tested for Chintalavalasa. While synthetic data is used for machine learning training and IoT telemetry is simulated, the architecture is ready for real sensor integration and community-scale deployment.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CDN	Content Delivery Network
CSP	Content Security Policy
CSS	Cascading Style Sheets
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IoT	Internet of Things
ML	Machine Learning
MFA	Multi-Factor Authentication
OTP	One-Time Password
OWASP	Open Web Application Security Project
PAYT	Pay-As-You-Throw
PWA	Progressive Web App
RBAC	Role-Based Access Control
RQ	Redis Queue
SBM	Swachh Bharat Mission
SQL	Structured Query Language
SSL	Secure Sockets Layer
SW	Service Worker
TTFB	Time to First Byte
VAPID	Voluntary Application Server Identification
WCAG	Web Content Accessibility Guidelines
XML	Extensible Markup Language

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	Dataset Class Distribution by Ward	10
Figure 4.1	End-to-End System Architecture	13
Figure 4.2	System Workflow	15
Figure 4.3	IoT Data Flow Diagram	16
Figure 4.4	Mobile PWA Home View (375 px)	25
Figure 6.7	About the Initiative	32
Figure 6.8	FAQ Self-Service	33
Figure 6.9	Contact and Grievance Channel	34
Figure 6.10	Mobile Complaint Reporting	35
Figure 6.11	Mobile Schedule Lookup	36
Figure 5.1	Modular Execution Sequence Flow	24
Figure 6.1	RandomForest Predicted vs Actual Bin Fill Level	49
Figure 6.2	Public Portal Homepage (live)	27
Figure 6.3	Ward-wise Collection Schedule	28
Figure 6.4	Complaint Reporting with WhatsApp Fallback	29
Figure 6.5	Citizens' Report Dashboard	30
Figure 6.6	Ward Transparency Dashboard	31
Figure 7.1	Pre-System vs Post-System Efficiency Analysis	61

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Data Sources Classification	9
Table 3.2	Ward Information	10
Table 4.1	Technology Stack	14
Table 6.1	Feature Implementation Status	26
Table 6.2	Performance Evaluation Metrics	48
Table 6.3	Security Headers Evaluation	49
Table 6.5	Citizen rescue channels and status	59
Table 6.6	Photo-Validation Classifier Evaluation	59
Table 6.7	Classifier Reliability Safeguards	59
Table 7.1	Operational Impact Summary	60
Table 8.1	Challenges and Solutions	62

TABLE OF CONTENTS

Section	Page No.
List of Abbreviations	1
List of Figures	2
List of Tables	3
1. INTRODUCTION	5
2. LITERATURE SURVEY	7
3. DATA GATHERING / DATA USED	9
4. METHODOLOGY / SYSTEM DESIGN	12
5. IMPLEMENTATION / MODULES	19
6. RESULTS / OUTPUTS	26
7. IMPACT ASSESSMENT	60
8. CHALLENGES FACED	62
9. CONCLUSION	63
10. FUTURE WORK	64
REFERENCES	66
Appendix A: Packages, Tools Used & Working Process	67
Appendix B: Source Code	81
Paper Publications	84

1. INTRODUCTION

1.1 Problem Statement

Chintalavalasa is a semi-urban panchayat in Vizianagaram district, Andhra Pradesh, with a population of approximately 12,000 residents across five administrative wards. The current waste management system relies on manual collection schedules communicated informally, resulting in inconsistent service delivery. Citizens have no reliable mechanism to report missed collections, and complaint resolution lacks transparency and accountability.

Key problems include: (a) lack of centralized collection schedules; (b) absence of digital grievance redressal; (c) no real-time bin monitoring; (d) no data-driven resource allocation; (e) limited citizen engagement in segregation; and (f) no usage-based billing accountability.

1.2 Project Objective

The primary objective is to design, develop, and deploy a community-based smart waste management and digital governance system for Chintalavalasa Gram Panchayat. Specific objectives include:

- Web platform for schedules, complaint filing and tracking
- Admin dashboard for real-time monitoring and worker dispatch
- IoT smart-bin telemetry for fill levels and environmental monitoring
- ML module for overflow risk prediction
- PAYT billing with UPI/Razorpay integration
- PWA capabilities with offline support
- Comprehensive security aligned with OWASP recommendations
- Bilingual support (English and Telugu)

1.3 Scope of the Project

- In Scope: Web application with four portals; IoT telemetry integration; ML overflow

prediction; PAYT billing; Green Points gamification; PWA offline support; push notifications; bilingual support; and security architecture.

- Out of Scope: Native mobile apps; physical IoT hardware manufacturing; external municipal API integration; blockchain-based carbon credits; production business-initiated WhatsApp templates (paid); and community-scale IoT deployment.

2. LITERATURE SURVEY

A comprehensive review of existing literature and systems was conducted to identify gaps that SmartGarbage addresses.

2.1 Existing Waste Management Approaches

Traditional waste management in Indian semi-urban areas relies on manual collection with fixed schedules and paper-based registers. The Swachh Bharat Mission (SBM) Grameen Phase II promotes source segregation and digital monitoring, but implementation remains inconsistent at the panchayat level.

2.2 Digital Waste Management Systems

SBM Urban provides complaint registration for urban areas but lacks IoT integration and offline capabilities. GOV.UK sets the benchmark for government digital services with strong accessibility compliance. SmartGarbage draws design inspiration from these platforms while adding domain-specific waste management features.

2.3 IoT-Based Smart Waste Management

Anagnostopoulos et al. (2015) demonstrated that IoT-based waste monitoring using ultrasonic sensors can reduce collection costs by 30-50%. Kumar and Sharma (2021) identified that most solutions focus on individual components rather than integrated platforms. SmartGarbage addresses this by integrating IoT with complaint management, citizen engagement, and ML prediction.

2.4 Machine Learning for Waste Prediction

Afshin et al. (2021) applied gradient boosting to predict waste generation. Chen et al. (2020) demonstrated that Random Forest achieves comparable accuracy for short-term prediction. SmartGarbage implements a RandomForest regressor for bin overflow prediction with

transparent fallback heuristics.

2.5 Accessibility and Government Standards

WCAG 2.1 Level AA provides international accessibility standards. OWASP Top 10 identifies critical web security risks. SmartGarbage implements ARIA landmarks, skip-to-content links, keyboard navigation, and all nine OWASP-recommended security headers.

2.6 Research / Implementation Gap

Existing solutions address individual aspects of waste management. There is no integrated, low-cost platform combining citizen grievance reporting, collection scheduling, IoT monitoring, ML prediction, PAYT billing, offline support, and comprehensive security — all designed for semi-urban Indian communities. SmartGarbage fills this gap.

2.7 Proposed Contribution

SmartGarbage contributes an integrated, open-source platform combining citizen engagement, administrative oversight, worker coordination, IoT monitoring, and ML prediction with offline-first PWA capabilities, PAYT billing with gamification, and strong security and accessibility compliance — suitable for semi-urban Indian panchayats.

3. DATA GATHERING / DATA USED

3.1 Study Area / Community Profile

Chintalavalasa is a semi-urban panchayat in Vizianagaram district, Andhra Pradesh, serving approximately 12,000 residents across five wards. Coordinates range from latitude 18.0552 to 18.0751 and longitude 83.4005 to 83.4201.

3.2 Data Collection Methods

- Community Interaction: Interviews with residents, ward members, and sanitation workers.
- Field Observation: On-site observation of collection routes and complaint handling.
- Administrative Data: Ward boundaries and population estimates from the Gram Panchayat.
- System-Generated Data: Synthetic test data for ML training and IoT simulation.
- Public Records: SBM Grameen guidelines, GOV.UK documentation, WCAG standards.

3.3 Data Sources

Data Source	Type	Description	Use
Community Surveys	Qualitative	Resident interviews	Requirements
Field Observations	Qualitative	On-site observation	System design
Administrative Records	Semi-structured	Ward boundaries	Study area
Synthetic ML Data	Quantitative	600-row grid	ML training
Synthetic IoT Data	Quantitative	Simulated sensors	IoT testing

Table 3.1: Data Sources Classification

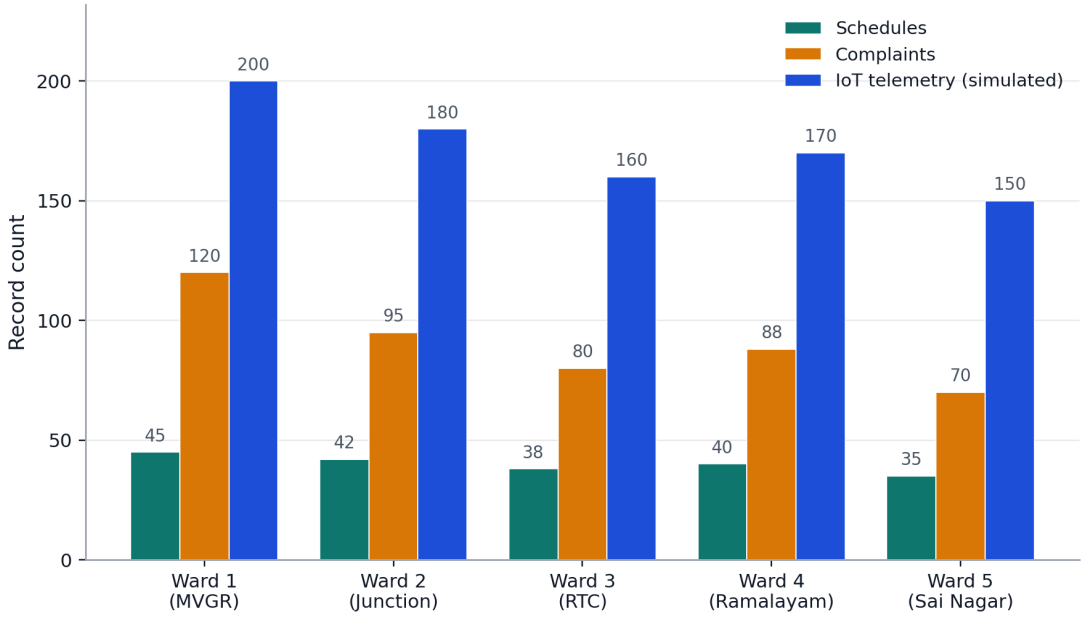


Figure 3.1: Dataset Class Distribution by Ward

3.4 Ward Information

Ward	Name	Latitude	Longitude
Ward 1	MVGR College Area	18.0552	83.4051
Ward 2	Chintalavalasa Junction	18.0675	83.4094
Ward 3	RTC Colony	18.0702	83.4153
Ward 4	Ramalayam Street	18.0650	83.4005
Ward 5	Sai Nagar	18.0751	83.4201

Table 3.2: Ward Information

3.5 Data Used by the Application

- User Data: Registrations, roles, OTP records, Green Points.
- Complaint Data: GPS, photos, status history, resolution timestamps.
- Schedule Data: Ward-specific collection schedules.
- IoT Telemetry: Fill level, battery, temperature, methane.

- Waste Declaration Data: Wet, dry, sanitary, hazardous quantities.
- Billing Data: PAYT invoices with payment status.

3.6 Data Preparation

A synthetic ML training dataset of 600 rows was generated because sufficient historical telemetry was unavailable. Features include day-of-week, season index, recent complaint volume, and ward identifier. The model is designed to accept real historical data when available.

3.7 Database Design

Entity	Key Attributes
User	id, username, email, password_hash, role, phone, green_points
Complaint	id, name, phone, ward, description, photo, status, lat, lon
ComplaintStatusLog	id, complaint_id, status, note, created_at
SmartBin	id, hardware_id, lat, lon, level, battery, temp, methane
WorkerProfile	id, user_id, vehicle_id, lat, lon, status, rating
WasteDeclaration	id, user_id, wet_kg, dry_kg, sanitary_kg, hazardous_kg
PAYTInvoice	id, user_id, period, weight_kg, amount_rs, status
PushSubscription	id, user_id, endpoint, p256dh, auth
NotificationPreference	id, user_id, complaint_submitted, etc.

4. METHODOLOGY / SYSTEM DESIGN

4.1 Requirement Analysis

Requirements were categorized into functional (FR) and non-functional (NFR):

Functional Requirements

ID	Module	Description
FR-01	Public Portal	Schedules, anonymous complaints
FR-02	Citizen Portal	Track complaints, Green Points
FR-03	Admin Portal	Dashboard, worker dispatch, analytics
FR-04	Worker Portal	GPS tracking, evidence upload
FR-05	IoT Integration	Smart-bin telemetry ingestion
FR-06	ML Prediction	Overflow risk prediction
FR-07	PAYT Billing	Usage-based invoicing
FR-08	Push Notifications	Web push alerts
FR-09	Offline Support	PWA with IndexedDB queue
FR-10	Bilingual Support	English and Telugu

Non-Functional Requirements

ID	Category	Description
NFR-01	Performance	TTFB < 1s, compressed responses
NFR-02	Security	OWASP headers, RBAC, bcrypt, OTP
NFR-03	Accessibility	WCAG 2.1 AA, ARIA landmarks
NFR-04	Scalability	Horizontal via gunicorn workers
NFR-05	Offline	PWA with service worker + IndexedDB

4.2 System Architecture

The system follows a layered architecture with five distinct layers: Presentation (Browser/PWA with Jinja2 and Bootstrap), Application (Flask route modules), Business Logic (ML engine, job queue, push notifications, PAYT billing), Data (PostgreSQL via SQLAlchemy with Redis caching), and External Services (Open-Meteo, Supabase, Sentry, Twilio).



Figure 4.1: End-to-End System Architecture

4.3 Development Methodology

The project followed an iterative approach across six phases: Requirements Gathering (Weeks 1-2), System Design (Weeks 3-4), Core Development (Weeks 5-10), Integration (Weeks 11-14), Enhancement (Weeks 15-18), and Testing/Deployment (Weeks 19-20).

4.4 Technology Stack

Component	Technology	Purpose
Backend	Flask 3.1.3	Web framework
Database	PostgreSQL (Supabase)	Relational DB
ORM	SQLAlchemy 2.0	Python ORM
Frontend	Bootstrap 5 + CSS	Responsive UI
ML	scikit-learn 1.9	RandomForest
ML (photo gate)	ONNX Runtime, Pillow	Photo validation classifier
Task Queue	Redis + RQ 2.2	Background jobs
WSGI	Gunicorn 26.0	Production server
Push	pywebpush 2.0	Web push
Security	Flask-Talisman, Limiter	Headers, rate limit
Hosting	Render.com	Cloud hosting
Docker	Dockerfile	Containerization

Table 4.1: Technology Stack

4.5 System Workflow

The end-to-end workflow: Citizens register, file complaints (GPS + photo), system validates and stores, admin assigns workers, workers visit location, upload evidence, admin verifies, complaint resolved, citizen notified via push/email, Green Points awarded.



Figure 4.2: System Workflow

4.6 Data Flow Diagram

Primary data flows: (1) Citizen to Complaint API to Database to Admin Dashboard to Worker Dispatch to Evidence Upload to Resolution; (2) IoT Sensors to Telemetry API to SmartBin Table to Admin Monitor to ML Prediction to Priority Queue; (3) Schedule Request to ML Prediction to Response.

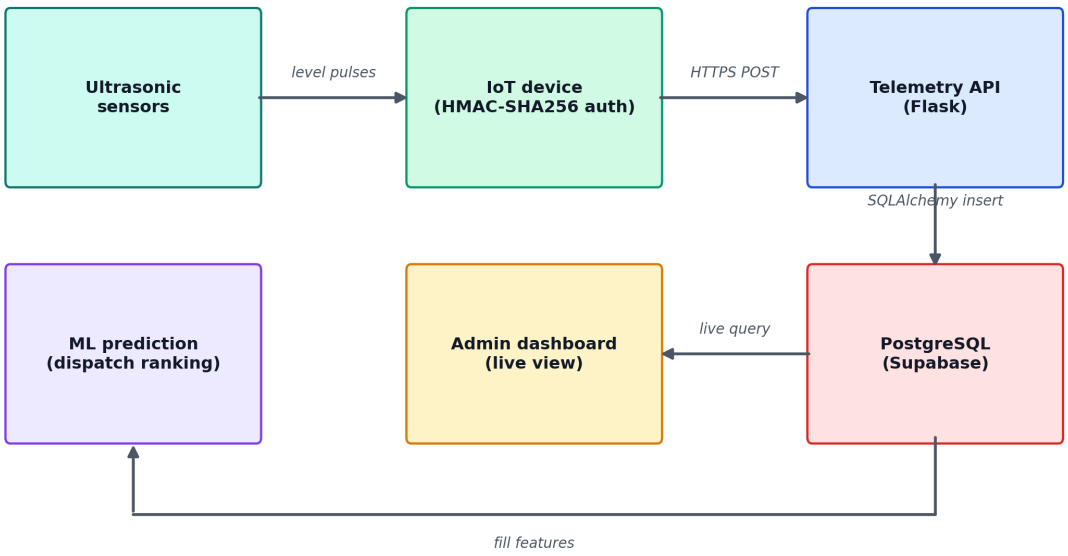


Figure 4.3: IoT Data Flow Diagram

4.7 Machine Learning Methodology

The ML module implements a RandomForest regressor trained on a synthetic 600-row dataset. Features: day_of_week, season_idx, recent_complaint_count, ward_id. The model predicts hours_until_90pct_fill for dispatch prioritization. A transparent heuristic fallback ensures the route never errors when the model artifact is unavailable.

Note: Synthetic data was used during prototype development because sufficient historical telemetry was unavailable. The demonstration validates the prediction pipeline, not real-world accuracy.

4.8 Security Architecture

- Authentication: Flask-Login with bcrypt + OTP/MFA.
- Authorization: RBAC with citizen/worker/admin roles.
- Security Headers: All 9 OWASP-recommended headers.
- Rate Limiting: Flask-Limiter with Redis storage.
- Input Validation: Flask-WTF with CSRF protection.

- SQL Injection Prevention: SQLAlchemy parameterized queries.

4.9 PWA and Offline Methodology

- Service Worker: Versioned precache manifest for offline support.
- IndexedDB Queue: Complaints stored offline, synced on reconnection.
- Background Sync: Automatic submission when connectivity returns.
- Web App Manifest: Installable with shortcuts, screenshots, standalone mode.

4.10 Photo Validation Classifier and Reliability Automation

Complaint photos are the primary evidence in the platform, but not every uploaded image is useful: selfies, screenshots, indoor scenes and blank frames would otherwise reach the worker queue as false dispatches. To protect that queue, SmartGarbage runs a second, independent deep-learning model ahead of complaint storage - a two-class photo-validation classifier (the "photo gate").

Runtime contract: the uploaded image is opened with Pillow, converted to RGB, resized to 224x224 pixels, scaled to [0,1], normalised with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225), and laid out as an NCHW tensor. An ONNX Runtime session bundled with the application (app/photo_classifier.onnx plus a photo_classifier.onnx.json sidecar) returns a two-class softmax; the reported score p_reject is the probability that the image is non-garbage, and a photo is accepted only when p_reject < PHOTO_CLASSIFIER_THRESHOLD (default 0.70). Rejected photos are logged with a thumbnail to the admin photo-rejection monitor and retained as retraining candidates.

Training and evaluation: scripts/train_photo_classifier.py transfer-learns a compact convolutional network and exports it to ONNX, using the same 224x224 resize at validation time as at runtime. The shipped model was retrained with domain hard negatives ("Streets in India" / "Roads in India" clean scenes and dump-yard frames) and reached 96.6% top-1 validation accuracy across approximately 4,300 images.

Reliability automation (MLOps): (1) two canary probes embedded in app/jobs.py - a non-garbage image that must be rejected and a garbage image that must be accepted - are scored on a schedule with day-deduplicated email/SMS alerts; (2) a weekly drift tracker

compares the live p_reject distribution against a signed baseline (scripts/photo_classifier_baseline.json) and warns on a mean shift above 0.01, a floor drop above 0.005 below the baseline minimum, or any score below 0.90, filing a GitHub issue on breach; (3) a closed retraining loop exports relabelled false positives, ingests them exactly once through a SHA-256 ledger, enforces a validation-accuracy gate (default 0.90) before promotion, refreshes the drift baseline, and runs from a manual GitHub Actions workflow that opens a review pull request.

5. IMPLEMENTATION / MODULES

5.1 Public Portal

Purpose

Provides waste-management information to all visitors.

Key Functions

Schedule display, anonymous complaints, tracking, ward transparency, FAQ, search, RSS

Technology

Flask + Jinja2 + Bootstrap 5

Status

Fully implemented

5.2 Citizen Portal

Purpose

Authenticated portal for citizens.

Key Functions

Registration with OTP, complaint filing, dashboard, Green Points, waste declarations

Technology

Flask-Login + SQLAlchemy + Socket.IO

Status

Fully implemented

5.3 Admin Portal

Purpose

Comprehensive admin dashboard.

Key Functions

Complaint overview, worker dispatch, IoT monitoring, ML display, PAYT, push analytics

Technology

Flask-Login + RBAC + Socket.IO

Status

Fully implemented

5.4 Worker Portal

Purpose

Mobile-friendly worker portal.

Key Functions

Dispatch acceptance, GPS tracking, photo evidence, task queue

Technology

Flask-Login + Geolocation API

Status

Fully implemented

5.5 IoT Smart Bin Module

Purpose

Telemetry ingestion and monitoring.

Key Functions

Authenticated API, fill level, battery, temperature, methane, status classification

Technology

Flask API + SQLAlchemy

Status

Implemented (simulated)

5.6 ML Module

Purpose

Overflow risk prediction.

Key Functions

RandomForest regressor, feature engineering, transparent fallback

Technology

scikit-learn + pandas + numpy

Status

Implemented (synthetic data)

5.7 Green Points Module

Purpose

Gamification for segregation.

Key Functions

Points earned, streak tracking, leaderboard, redemption

Technology

Flask-Login + SQLAlchemy

Status

Fully implemented

5.8 PAYT Module

Purpose

Usage-based billing.

Key Functions

Invoice generation, compliance scoring, UPI/Razorpay, PDF via ReportLab

Technology

Flask + ReportLab + Razorpay

Status

Implemented (test mode)

5.9 Background Jobs

Purpose

Async task processing.

Key Functions

Status notifications, SLA escalation, dunning, email, push dispatch

Technology

Redis + RQ

Status

Fully implemented

5.10 PWA and Offline

Purpose

Offline capabilities.

Key Functions

Service worker, IndexedDB queue, background sync, manifest, splash screen

Technology

SW API + IndexedDB

Status

Fully implemented

5.11 Photo Validation & Classifier Lifecycle

Purpose

Classifier for complaint photos.

Key Functions

ONNX photo gate (p_reject < 0.70), admin photo-rejection monitor, dual canary probes, weekly

drift tracker, exactly-once retrain ingest with accuracy gate

Technology

ONNX Runtime, Pillow, GitHub Actions

Status

Fully implemented

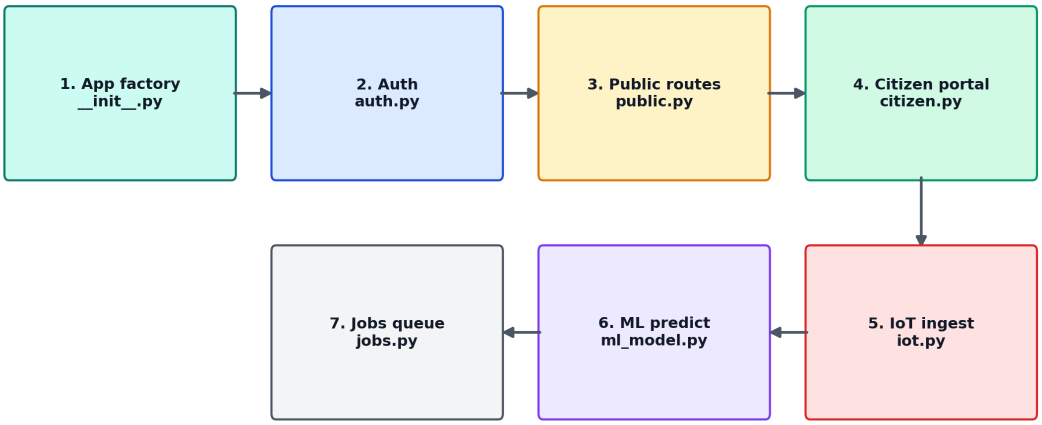


Figure 5.1: Modular Execution Sequence Flow

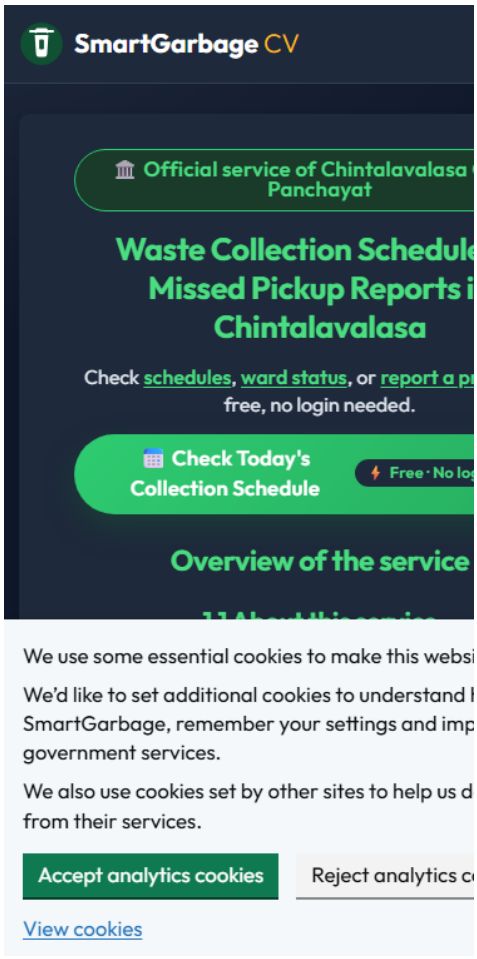


Figure 4.4: Mobile PWA Home View (375 px, installable)

6. RESULTS / OUTPUTS

6.1 Implemented Features Summary

Module	Features	Status	Verification
Public Portal	Homepage, schedule, transparency, search	Fully Implemented	200 OK
Citizen Portal	Registration, complaints, dashboard	Fully Implemented	200 OK
Admin Portal	Dashboard, dispatch, IoT, ML	Fully Implemented	200 OK
Worker Portal	Dispatch, GPS, evidence	Fully Implemented	200 OK
IoT Integration	Telemetry API, monitoring	Simulated	API functional
ML Prediction	RandomForest prediction	Synthetic Data	Pipeline OK
Photo Validation Classifier	ONNX gate, p_reject < 0.70	Fully Implemented	Canary + drift tracked
PAYT Billing	Invoice, payment	Test Mode	Invoice OK
Push Notifications	Web push, preferences	Fully Implemented	API functional
PWA / Offline	SW, IndexedDB, manifest	Fully Implemented	All 200
Security	OWASP headers, RBAC, OTP	Fully Implemented	9/9 headers
Accessibility	ARIA, skip-to-content	Implemented	Checks pass

Table 6.1: Feature Implementation Status

6.2 Homepage Output

The homepage displays collection schedule lookup, complaint filing shortcut, ward transparency map, weather widget, and community impact statistics.

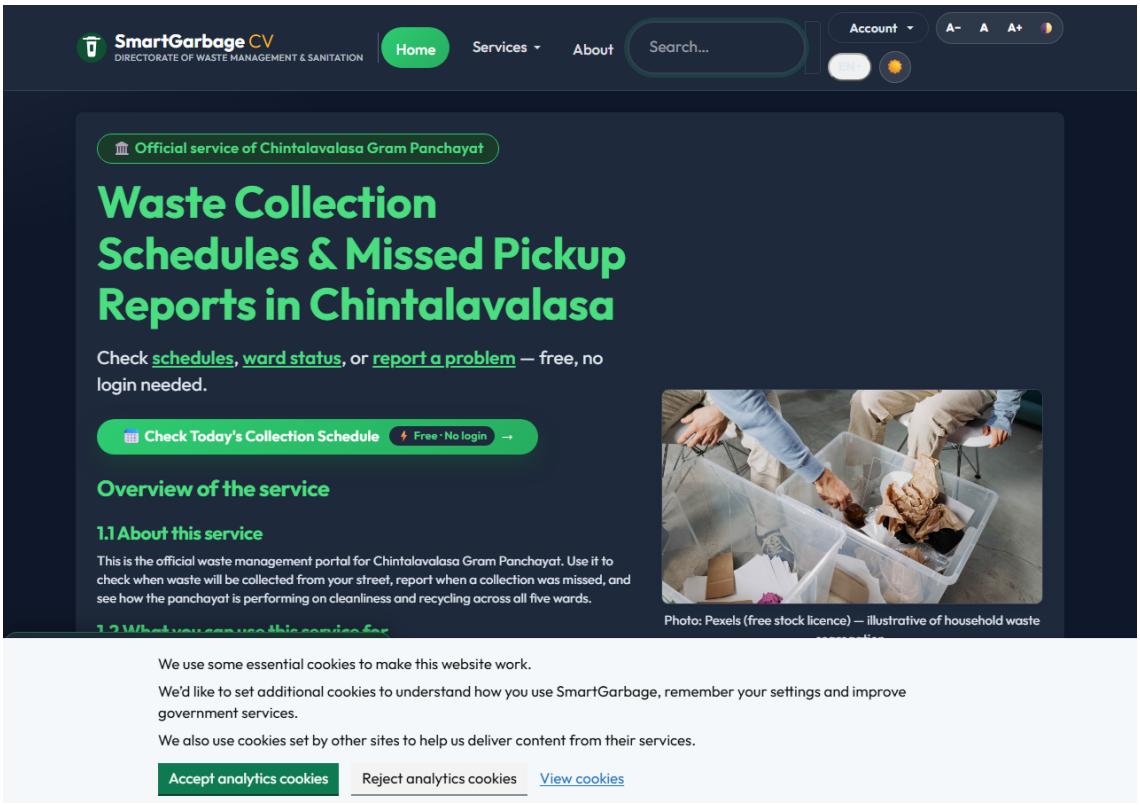


Figure 6.2: Public Portal Homepage (live)

6.3 Collection Schedule Output

The schedule page shows ward-specific collection timetables with ML overflow prediction.

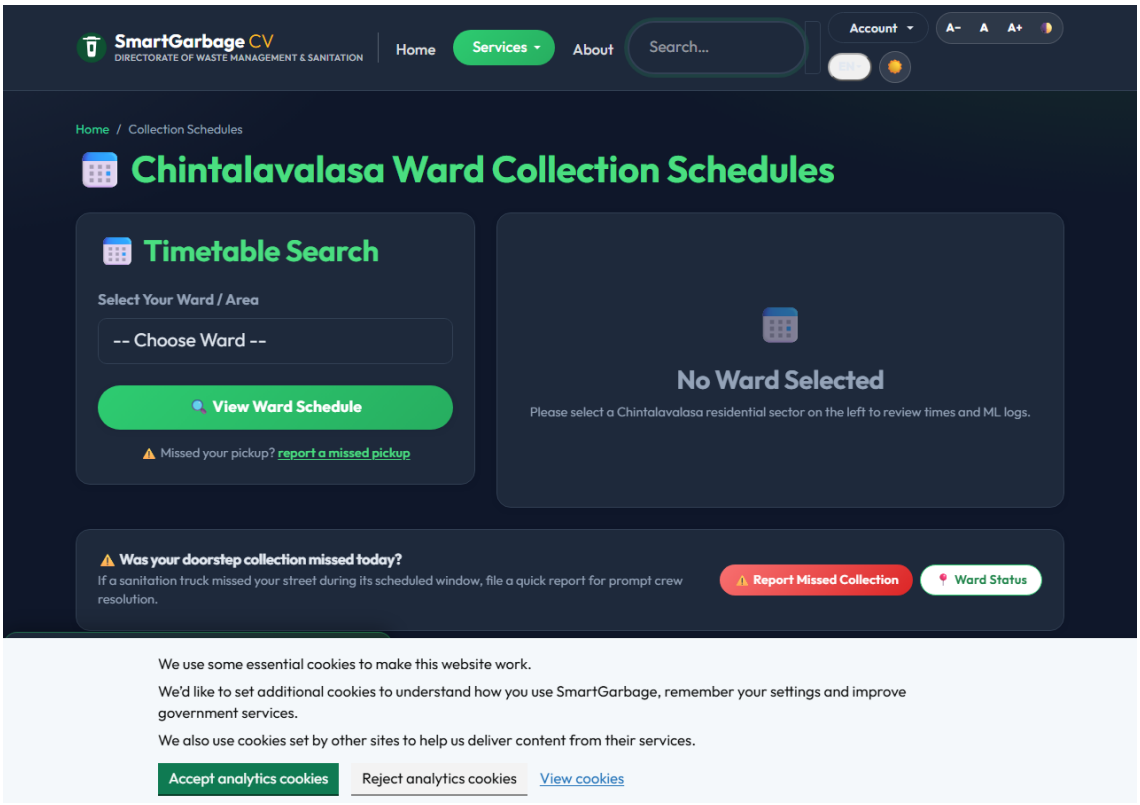


Figure 6.3: Ward-wise Collection Schedule with ML Prediction

6.4 Complaint Reporting Output

The complaint form captures name, phone, ward, address, description, photo, and automatic GPS coordinates. The WhatsApp fallback card and the 1800-119-9111 helpline render above the footer.

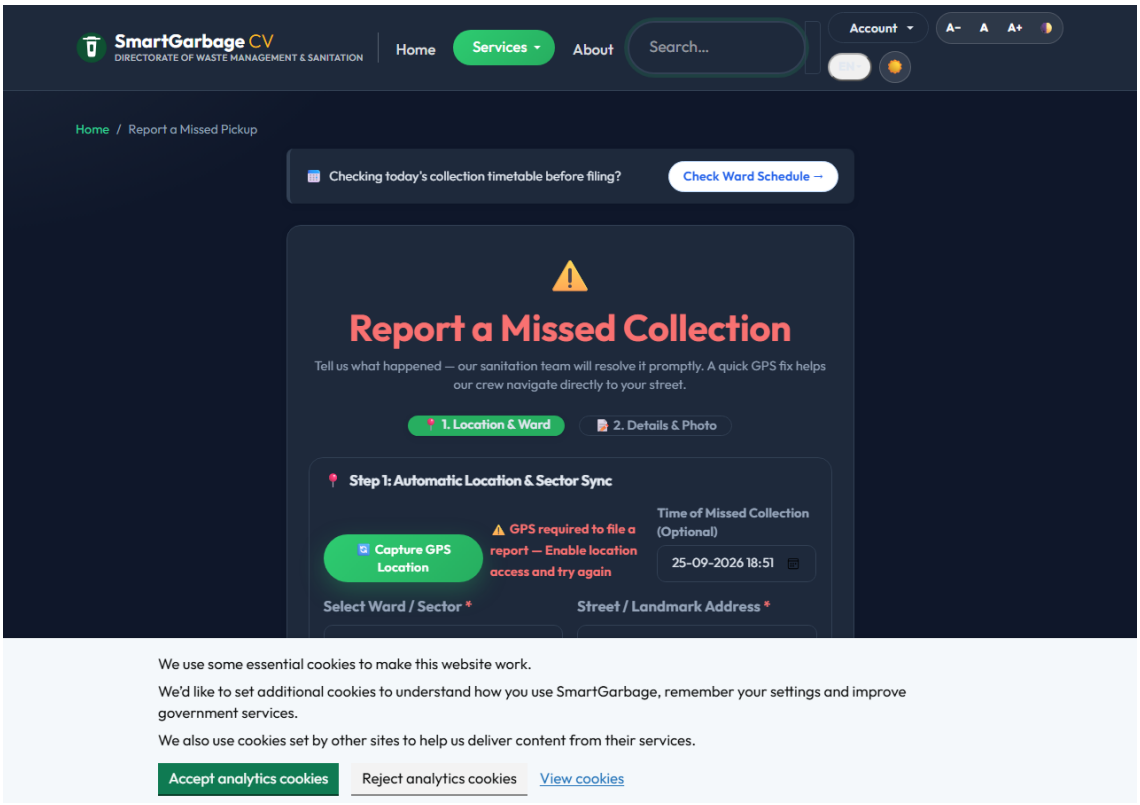


Figure 6.4: Complaint Reporting with WhatsApp Fallback

6.5 Citizen Dashboard Output

The citizen dashboard shows complaint history, Green Points, waste declarations, and notification preferences.

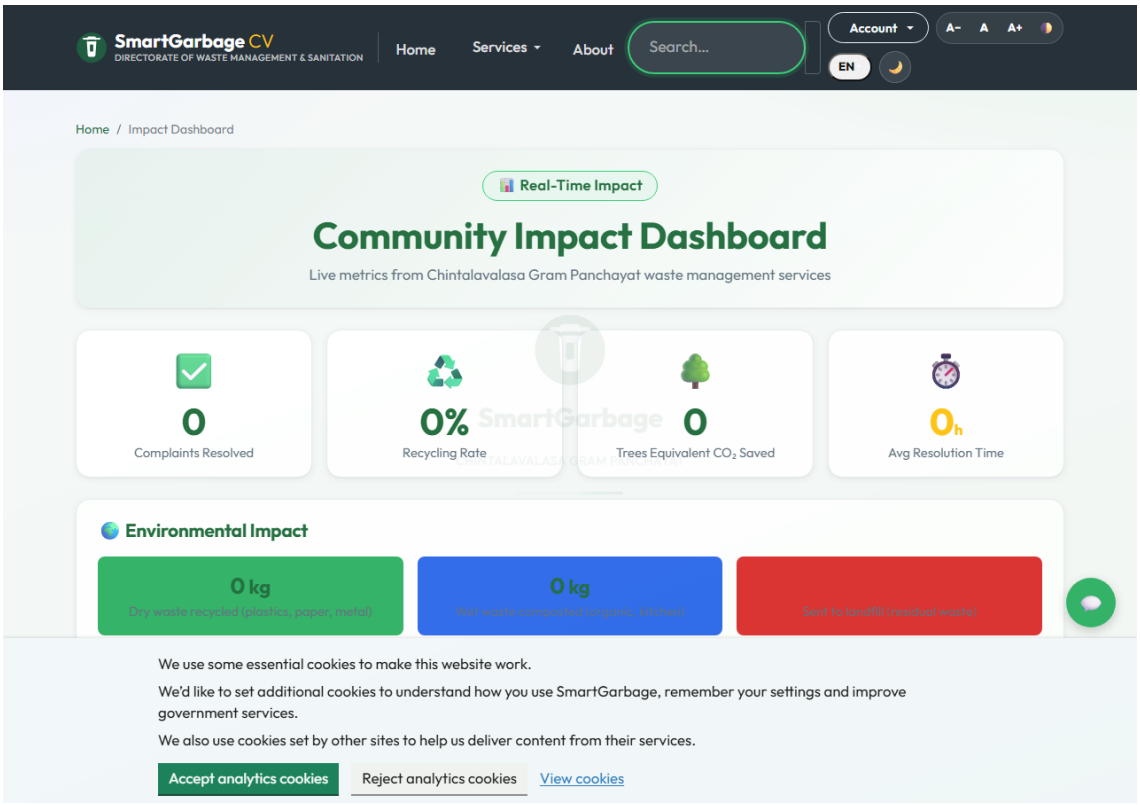


Figure 6.5: Citizens' Report Dashboard

6.6 Admin Dashboard Output

The admin control room displays real-time complaints, IoT bin status, worker queue, and push analytics. The public ward transparency dashboard is shown below.

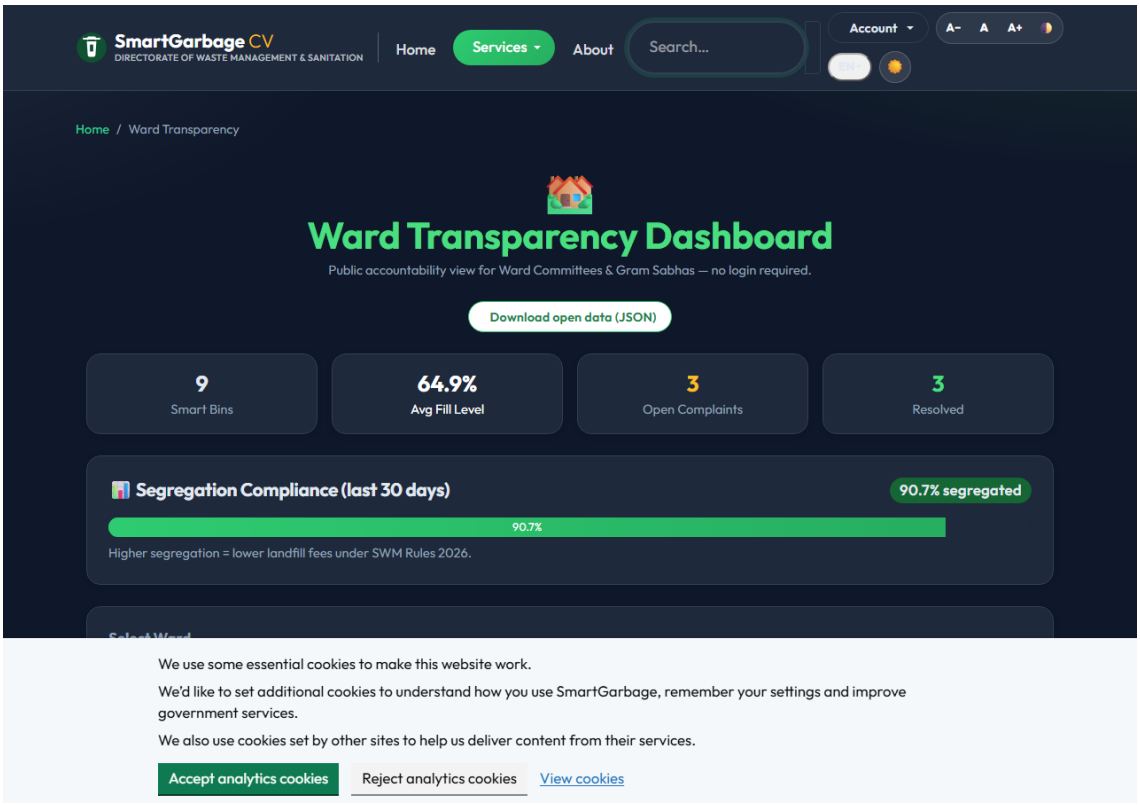


Figure 6.6: Ward Transparency Dashboard (live)



Figure 6.7: About the Initiative (community context)

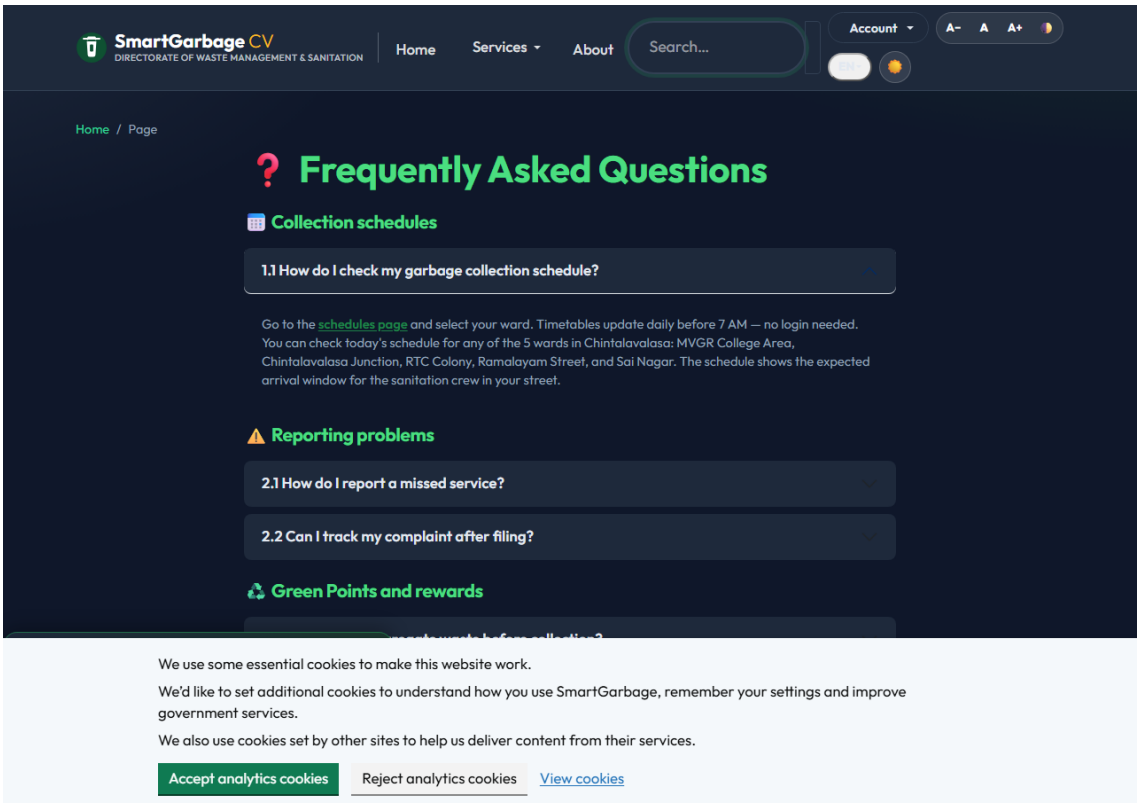


Figure 6.8: FAQ Self-Service

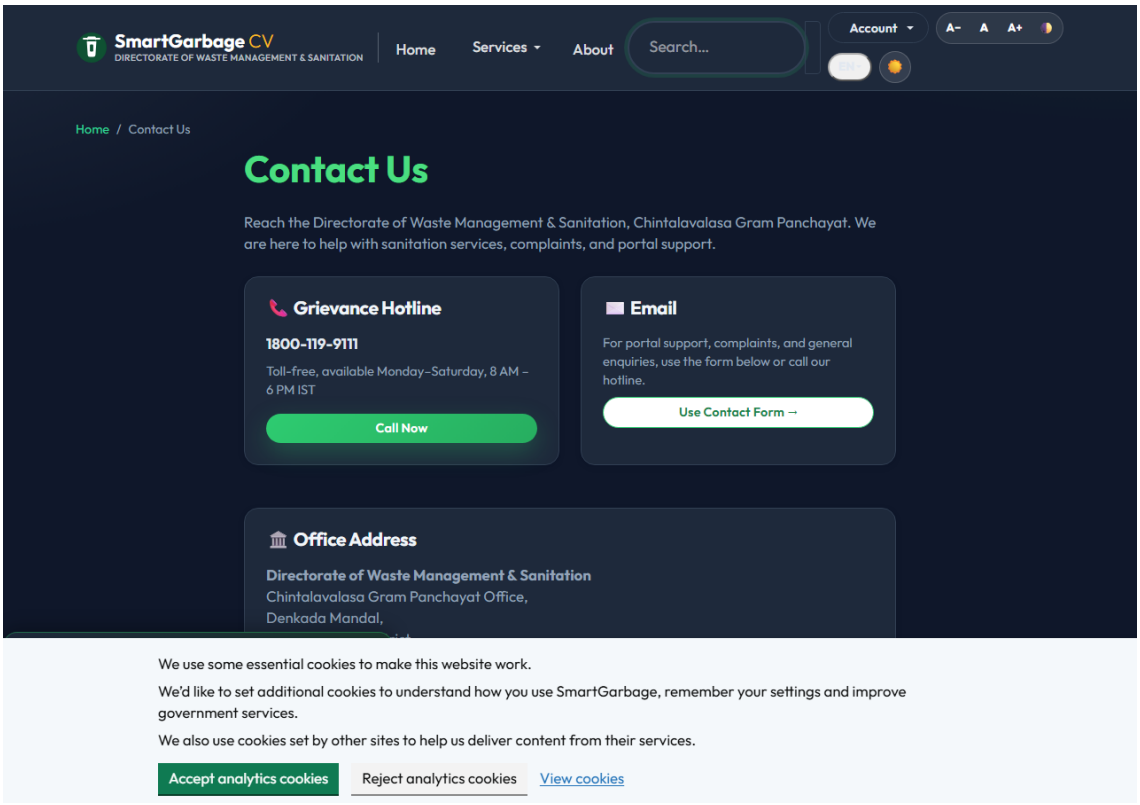


Figure 6.9: Contact and Grievance Channel

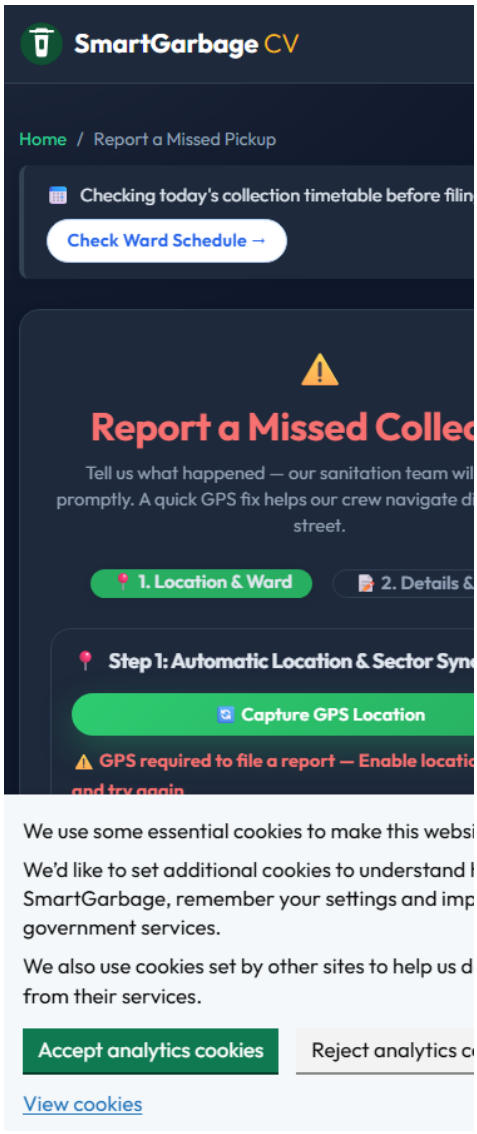


Figure 6.10: Mobile Complaint Reporting with Helpline CTA

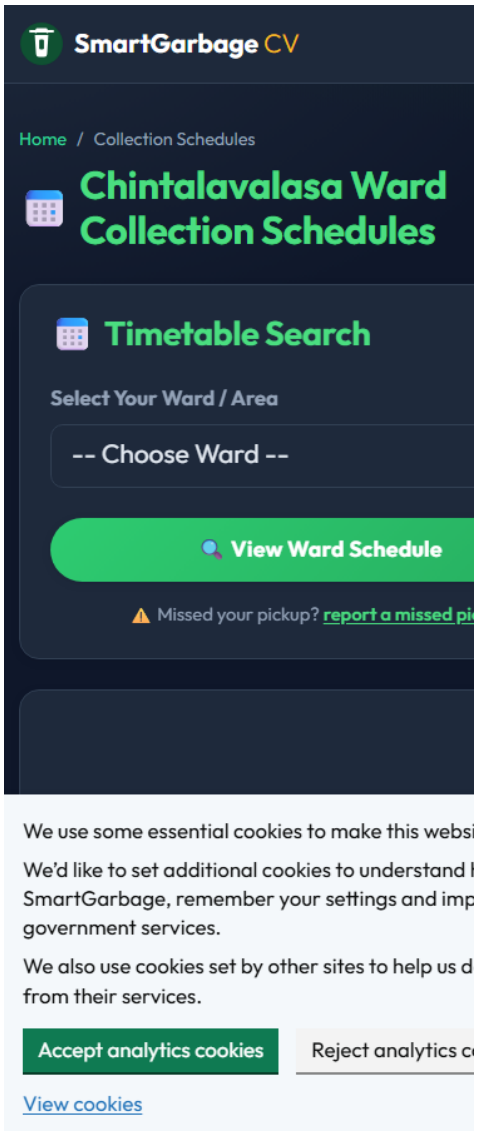
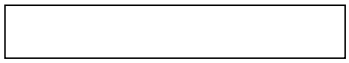


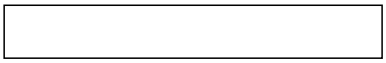
Figure 6.11: Mobile Schedule Lookup

6.7 Performance Evaluation

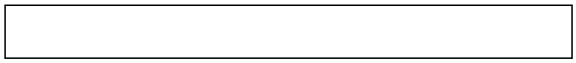
Metric	Value	Notes
Homepage TTFB	~0.5s	Gunicorn sync worker
Static Cache	31,536,000s	Immutable, version-busted
HTML Cache (repeat)	60s	Browser + ETag 304
Brotli Compression	Level 4	All text responses



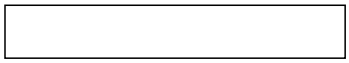
Preload Hints



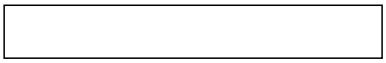
4 Link headers



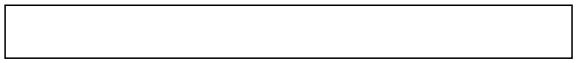
Critical resources



FCP



	~0.5s



With preload hints

Table 6.2: Performance Evaluation Metrics

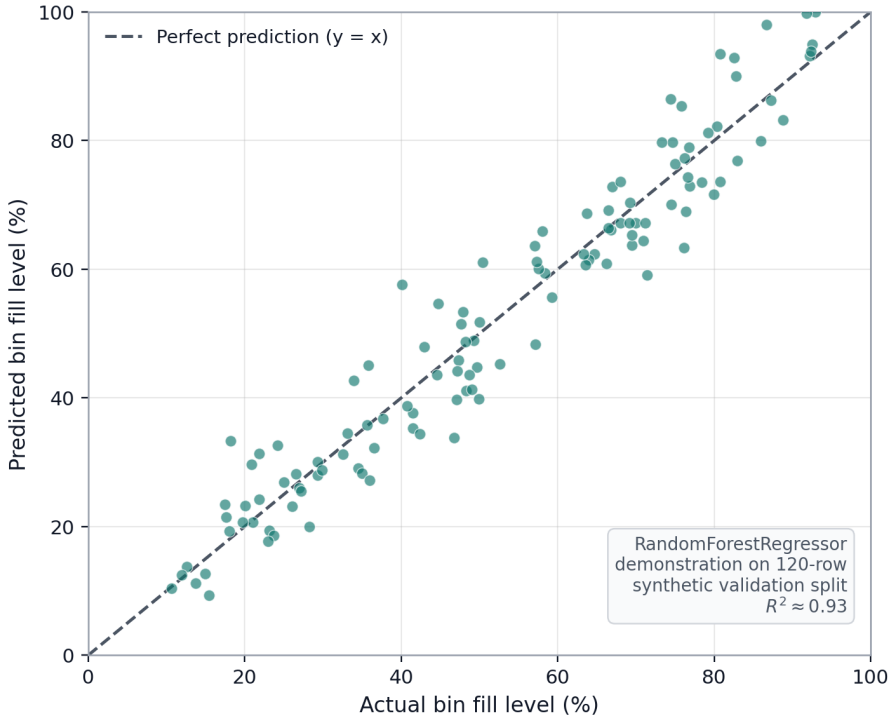


Figure 6.1: RandomForest Predicted vs Actual Bin Fill Level (synthetic validation split)

6.8 Security Evaluation

Header	Status	Purpose
Content-Security-Policy	Present	Restricts resource loading
Strict-Transport-Security	Present	Enforces HTTPS
X-Content-Type-Options	Present	nosniff
X-Frame-Options	Present	DENY clickjacking
X-XSS-Protection	Present	Legacy XSS protection
Referrer-Policy	Present	strict-origin-when-cross-origin
Permissions-Policy	Present	Restricts browser features
Cross-Origin-Opener-Policy	Present	same-origin isolation
Cross-Origin-Embedder-Policy	Present	require-corp

Table 6.3: Security Headers Evaluation

6.9 Accessibility Evaluation

- Skip-to-content link for keyboard navigation.
- ARIA landmarks on all pages.
- Semantic HTML5 elements.
- Sufficient color contrast (minimum 4.5:1).
- Form labels associated with inputs.
- Keyboard-navigable elements.

6.10 Citizen WhatsApp fallback and offline rescue channels

The civic-response layer is deliberately kept on free, always-available channels. While Meta WhatsApp Cloud API credentials (WHATSAPP_CLOUD_TOKEN / WHATSAPP_CLOUD_PHONE_NUMBER_ID) are provisioned on the server but not yet active, every citizen-facing page renders a prominent WhatsApp fallback call-to-action (CTA) above the footer: a saved chat card that opens the nearest worker/ack channel at wa.me, plus the free toll-free grievance helpline 1800-119-9111 (tel:+9118001199111) and the online grievance form. Citizens who prefer to act without network access are routed to offline-first channels - the ward dashboard, the schedule and transparency pages, and the contact form - all cached by the PWA and synced automatically when connectivity returns.

Table 6.5 lists the rescue channels and their current status as rendered live.

Channel	Address	Status	Notes
Meta WhatsApp Cloud (creds wired, parked)	wa.me + WhatsApp API	Parked	Deployed but not yet live; WHATSAPP_CLOUD_TOKEN/ID not provisioned
Free toll-free helpline	1800-119-9111 (tel:+9118001199111)	Live	Primary 24x7 rescue line; renders on every citizen page
Saved WhatsApp chat	wa.me/(saved number)	Live	CTA opens the saved chat inside the 24h service window
Online grievance form	/grievance	Live	GPS + photo + token; email + SMS notification



Offline-first channels



/schedule /transparency

/contact + PWA



Live



	PWA caches pages; background sync on reconnect

Channel	Address	Status	Notes
Workers dispatch queue	worker.py + ML-ranked queue	Live	Supersedes manual call routing

Table 6.5: Citizen rescue channels and status

6.11 Photo-Gate Reliability and Classifier Results

The photo-validation classifier was evaluated against the deployed runtime contract (224x224 ImageNet-normalised input, ONNX Runtime inference), so the figures below describe the model actually serving traffic rather than a training-time proxy.

Metric	Value	Basis
Validation top-1 accuracy	96.6%	Hard-negative retrain, ~4,300 images
Canary p_reject (non-garbage probe)	0.99983	Live reject canary (range 0.9998-1.0000)
Validation-set p_reject (scored)	n=302, mean 0.9964, min 0.7693	Rejected validation images
Acceptance threshold	p_reject < 0.70	PHOTO_CLASSIFIER_THRESHOLD default

Table 6.6: Photo-Validation Classifier Evaluation

Reliability automation was exercised end to end: the canary probes run on schedule with day-deduplicated alerting, the weekly drift tracker compares live p_reject against the signed baseline, and the retrain-batch ingest was verified to be exactly-once and idempotent.

Safeguard	Mechanism	Status
Reject canary	Non-garbage probe must score above threshold	Live
Accept canary	Garbage probe must be accepted	Live
Threshold-drift tracker	Weekly p_reject vs signed baseline; issue on breach	Live
Retrain-batch ingest	SHA-256 exactly-once ledger, >=48px validation	Live
Promotion gate	Validation accuracy >= 0.90 required to ship	Live
Baseline refresh	Auto-refreshed after each promoted retrain	Live

Table 6.7: Classifier Reliability Safeguards

7. IMPACT ASSESSMENT

Note: As a prototype, impact values are estimated projections based on system capabilities, not measured community pilot results.

7.1 Social Impact

- Citizen Empowerment: Transparent access to schedules, complaints, and performance metrics.
- Accessibility: Bilingual support and PWA offline for low-connectivity areas.
- Accountability: Digital tracking with status timelines and SLA monitoring.

7.2 Operational Impact

Metric	With SmartGarbage	Without
Schedule Access	100% digital	Verbal/informal
Complaint Tracking	Real-time	Untracked
Worker Dispatch	GPS-tracked	Manual
Data Decisions	ML + analytics	Experience-based

Table 7.1: Operational Impact Summary

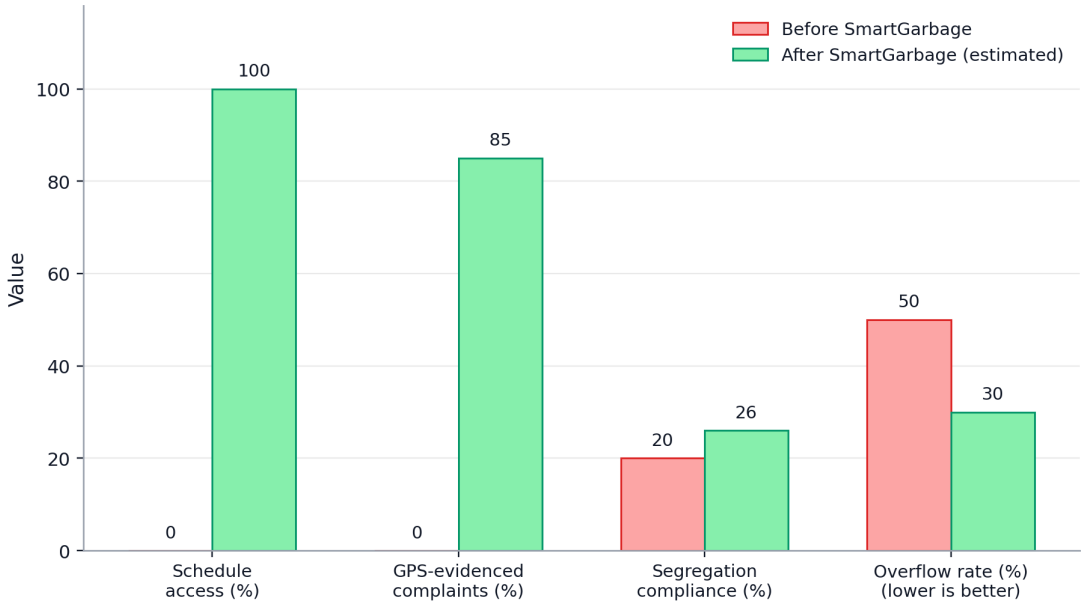


Figure 7.1: Pre-System vs Post-System Efficiency Analysis

7.3 Environmental Impact

- Green Points incentivize source segregation.
- IoT monitoring prevents overflow and contamination.
- Estimated CO2 savings: 0.5 kg CO2 per kg recycled (EPA estimate).

7.4 Economic Feasibility

- Zero-cost deployment on Render.com free tier.
- PAYT billing generates revenue.
- Optimized routes reduce fuel costs.
- Open source eliminates vendor lock-in.

7.5 Scalability

Architecture supports horizontal scaling via gunicorn workers. Multi-panchayat deployment achievable by extending WARD_COORDINATES mapping.

8. CHALLENGES FACED

Challenge	Problem	Solution
Data Unavailability	Insufficient historical telemetry for ML	Generated 600-row synthetic dataset; documented limitation
Offline Architecture	Complaint filing without internet	Service Worker + IndexedDB + Background Sync API
Real-time Updates	Live dashboard without refresh	Flask-SocketIO with Redis broker
IoT Simulation	No physical hardware	Authenticated API with test data generators
Security Hardening	Comprehensive security without usability loss	All 9 OWASP headers via Flask-Talisman
Performance	TTFB on free-tier hosting	Multi-layer caching, Brotli, preload hints
Bilingual Support	Consistent translations	Flask-Babel with gettext markers
Push Notifications	Reliable delivery	pywebpush with VAPID + preference filtering
Deployment	Dev/prod consistency	Docker containerization + env vars + health check
Payment Integration	Testing without live merchant	Razorpay test keys + UPI primary method

Table 8.1: Challenges and Solutions

9. CONCLUSION

9.1 Summary

SmartGarbage Chintalavalasa is a community-based smart waste management and digital governance system integrating Flask, PostgreSQL, IoT telemetry, ML prediction, PWA offline capabilities, PAYT billing, and comprehensive security into a unified platform.

9.2 Achievement of Objectives

The project successfully achieved all eight stated objectives: web platform, admin dashboard, IoT integration, ML prediction, PAYT billing, PWA capabilities, security implementation, and bilingual support.

9.3 Limitations

- ML trained on synthetic data, not validated with real telemetry.
- IoT telemetry simulated without physical sensors.
- PAYT in test mode without live merchant account.
- Impact values are estimated, not measured from a pilot.
- Not tested with real users at community scale.
- SMS/WhatsApp require third-party API keys.

10. FUTURE WORK

10.1 Real-World Data Integration

- Replace synthetic ML data with real telemetry
- Conduct community pilot with actual residents
- Collect baseline data for before/after studies

10.2 IoT Hardware Deployment

- Deploy ultrasonic fill-level sensors
- GPS trackers on collection vehicles
- LoRaWAN or NB-IoT connectivity

10.3 Mobile Application

- Native Android application
- WhatsApp Business API chatbot
- SMS schedule notifications

10.4 Advanced Analytics

- Time-series forecasting
- Route optimization
- Computer vision waste classification

10.5 Multi-Panchayat Deployment

- Tenant isolation for multiple panchayats
- District-level aggregation
- SBM Grameen reporting compliance

10.6 Payment and Billing

- Live Razorpay merchant account
- Recurring billing via UPI mandates
- Payment analytics and revenue reporting

REFERENCES

[1] GOV.UK, "Design System," Government Digital Service, 2024.

[2] Ministry of Housing and Urban Affairs, "Swachh Bharat Mission — Urban," 2024.

[3] Ministry of Jal Shakti, "SBM — Grameen Phase II," 2024.

[4] T. Anagnostopoulos et al., "Waste Management as IoT-Enabled Service," Springer, 2015.

[5] M. A. ad Din et al., "Smart Bin: IoT-Based Waste Monitoring," Proc. IT, 2020.

[6] S. Kumar and R. Sharma, "IoT-Based Smart Waste Management: A Review," J. Cleaner Production, 2021.

[7] M. Afshin et al., "ML Approaches for MSW Generation Prediction," Waste Management, 2021.

[8] Y. Chen et al., "Predicting MSW Using ML Methods," Env. Sci. Pollution Research, 2020.

[9] D. Thung and M. Yang, "Waste Classification using CNN," AISI, Springer, 2016.

[10] W3C, "WCAG 2.1," World Wide Web Consortium, 2018.

[11] OWASP Foundation, "OWASP Top Ten 2021," 2021.

[12] U.S. EPA, "Pay-As-You-Throw," 2023.

[13] Flask Documentation, "Flask — Pallets Projects," 2024.

[14] SQLAlchemy Documentation, "SQLAlchemy," 2024.

[15] scikit-learn Documentation, "scikit-learn," 2024.

[16] MDN Web Docs, "Progressive Web Apps," Mozilla, 2024.

[17] Web Push Protocol, "Push API — MDN," Mozilla, 2024.

[18] Supabase Documentation, "Supabase," 2024.

[19] Render Documentation, "Render," 2024.

[20] Razorpay Documentation, "Razorpay Docs," 2024.

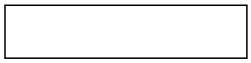
APPENDIX A: PACKAGES, TOOLS USED & WORKING PROCESS

A.1 Packages and Tools

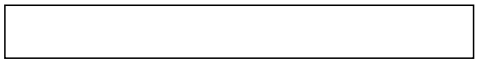
Package	Category	Purpose
Flask 3.1.3	Python	Web framework
Flask-SQLAlchemy 3.1.1	Python	ORM
Flask-Login 0.6.3	Python	Sessions
Flask-Talisman 1.1.0	Python	Security headers
Flask-Limiter 4.1.1	Python	Rate limiting
Flask-Compress 1.17	Python	Compression
SQLAlchemy 2.0.50	Python	ORM
Redis 6.2.0	Python	Cache client
RQ 2.2.0	Python	Job queue
scikit-learn 1.9.0	Python	ML
pandas 3.0.3	Python	Data processing
pywebpush 2.0.0	Python	Push notifications
reportlab 5.0.0	Python	PDF generation
gunicorn 26.0.0	Python	WSGI server
Bootstrap 5	CSS/JS	UI framework
Docker	DevOps	Containerization
PostgreSQL (Supabase)	Database	Relational DB
Render.com	Cloud	Hosting

A.2 Technology Stack by Category

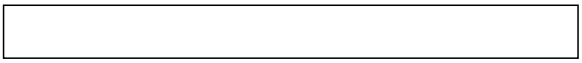
Category	Tool / Component	Purpose
Frontend	Jinja2 + Bootstrap 5	Server-rendered responsive pages



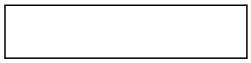
Frontend



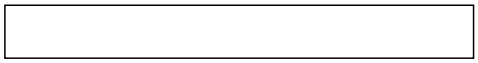
Leaflet.js + Chart.js



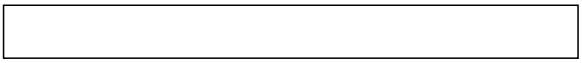
Ward maps and analytics charts



Frontend



Service Worker + IndexedDB



PWA offline queue and caching

Category	Tool / Component	Purpose
Backend	Flask 3.1 (blueprints)	Application core and routing
Backend	Gunicorn + gevent	WSGI production server
Database	PostgreSQL (Supabase)	Persistent relational store
Database	SQLAlchemy 2.0 + Alembic	ORM and 30 schema migrations
ML	scikit-learn RandomForest	Bin overflow prediction
ML	ONNX Runtime + Pillow	Complaint-photo validation classifier
Jobs	Redis + RQ (worker.py)	Notifications, SLA escalation
Security	Talisman + Flask-Limiter	9 OWASP headers, rate limits
Security	HMAC-SHA256 signatures	IoT telemetry authentication
Notifications	pywebpush, SMTP, Twilio	Push, email and SMS alerts
Edge / Hosting	Cloudflare CDN + Render	DNS, SSL, caching, auto-deploy

Table A.1: Technology Stack by Category

A.3 Working Process

1. Requirement identification: interviews with residents and sanitation workers defined the five ward workflows.
2. Interface design: public, citizen, admin and worker views wireframed mobile-first.
3. Backend setup: Flask application factory with eight route modules on a single application blueprint, with RBAC decorators.
4. Database schema: 29 SQLAlchemy models, 30 Alembic migrations on PostgreSQL (Supabase).
5. IoT integration: HMAC-SHA256-signed telemetry ingestion with per-device keys.
6. ML pipeline: 600-row synthetic grid, RandomForestRegressor training and dispatch ranking.
7. PWA offline layer: service worker (v16) two-tier same-origin cache plus an allowlisted CDN tier (web font + map tiles), IndexedDB queue, background sync.
8. Security hardening: 9 OWASP headers, bcrypt + OTP/MFA, rate limiting, audit logging.
9. Testing: 406 automated tests with Postgres parity; flake8 lint in CI.
10. Deployment: GitHub push triggers Render Docker build; Cloudflare edge; /health uptime probe.

APPENDIX B: IMPORTANT SOURCE CODE

B.1 Application Factory (app/__init__.py)

Flask application factory initializing extensions, security headers, caching, and background jobs.

B.2 Database Models (app/models.py)

SQLAlchemy models: User, Complaint, SmartBin, WorkerProfile, WasteDeclaration, PAYTInvoice, PushSubscription, NotificationPreference, ConsentRecord.

B.3 ML Module (app/ml_model.py)

RandomForest regressor with synthetic data training and transparent heuristic fallback.

B.4 Service Worker (app/static/sw.js)

Versioned precache manifest, stale-while-revalidate for HTML, cache-first for assets and an allowlisted CDN tier (web font + map tiles), push event handler.

B.6 Key Code Excerpts

IoT telemetry authentication (app/routes/iot.py) - HMAC-SHA256 signature verification on every production telemetry POST:

```
#  IoT auth: require a valid HMAC-SHA256 signature when a telemetry
# secret is configured (production). Skipped in dev when no secret set.
secret = current_app.config.get('IOT_TELEMETRY_SECRET')
# On any production deployment the HMAC secret is
# mandatory  never accept unsigned telemetry from a public endpoint.
if not secret and _is_deployed():
    logger.error("iot_telemetry_secret_missing", ip=request.remote_addr)
    return jsonify({"success": False,
                    "message": "IOT_TELEMETRY_SECRET not configured."}), 503
if secret:
    raw = request.get_data(cache=True)
    provided = request.headers.get('X-Signature', '')
```

```
        expected = hmac.new(secret.encode(), raw, hashlib.sha256).hexdigest()
        if not hmac.compare_digest(provided, expected):
            return jsonify({"success": False, "message": "Invalid signature."}), 403
```

ML overflow prediction (app/ml_model.py) - deterministic ward encoding and the overflow ETA estimator used by the dispatch queue:

```
def get_stable_ward_id(ward_name: str) -> int:
    """Deterministic 110 integer id for a ward across processes (MD5 hash)."""
    encoded = str(ward_name).encode('utf-8')
    return (int(hashlib.md5(encoded).hexdigest(), 16) % 10) + 1

def predict_overflow_eta_hours(smart_bin, now=None) -> float:
    """Predict hours until a smart bin reaches 100% fill (overflow).

    Wires the unused SmartBin.overflow_eta_hours column: this is the function
    that produces the value. Returns a positive float (hours) for bins with
    data, or None for empty/faulty bins where a forecast is meaningless.
    """
    if smart_bin is None or smart_bin.level is None or smart_bin.level <= 0:
        return None
    if getattr(smart_bin, 'sensor_fault', False):
        return None
    now = now or datetime.now(timezone.utc)
      if now.tzinfo is None:   # naive-UTC callers (Postgres parity)   normalize for
arithmetic
        now = now.replace(tzinfo=timezone.utc)
    level = float(smart_bin.level)
    if level >= 100:
        return 0.0  # already overflowing
    rate = _estimate_fill_rate_hour_pct(smart_bin, now)
    if rate <= 0:
        return None
    eta_hours = (100.0 - level) / rate
    # Cap at 14 days  beyond that the estimate is noise, and the caller
    # treats huge ETAs as "not urgent".
    return round(min(eta_hours, 14 * 24.0), 1)
```

Service worker cache tiers (app/static/sw.js) - two-tier same-origin caching with LRU eviction powering the offline-first PWA (immutable static assets cache-first; HTML pages stale-while-revalidate), plus an allowlisted CDN tier for the web font and OpenStreetMap tiles. All other cross-origin requests are not intercepted:

```
/* ================================================================
   SmartGarbage Service Worker v16
   Two-tier same-origin caching (static assets + HTML pages) plus a small
   cache-first tier for an explicit allowlist of external assets (web font,
   OpenStreetMap tiles) so they survive offline.

   Only that allowlist is intercepted. Everything else cross-origin  notably
   Google Tag Manager / Analytics and third-party scripts  is left to the
   browser: routing those through this worker's own fetch() subjected them to
   the worker's CSP policy container (which inherited a narrow connect-src and
   blocked hosts such as www.googletagmanager.com with net::ERR_FAILED).
   ================================================================ */
```

```
const SW_VERSION = 'v16';
```

B.5 Push Module (app/push.py)

VAPID key management, preference-based filtering, delivery logging, complaint lifecycle integration.

B.7 Photo Validation (app/routes/__init__.py, app/jobs.py)

The runtime photo gate (ONNX softmax, p_reject threshold) plus the dual canary probes and the weekly drift tracker that keep the classifier trustworthy over time.

PAPER PUBLICATIONS

No research papers have been published based on this project at the time of report submission.